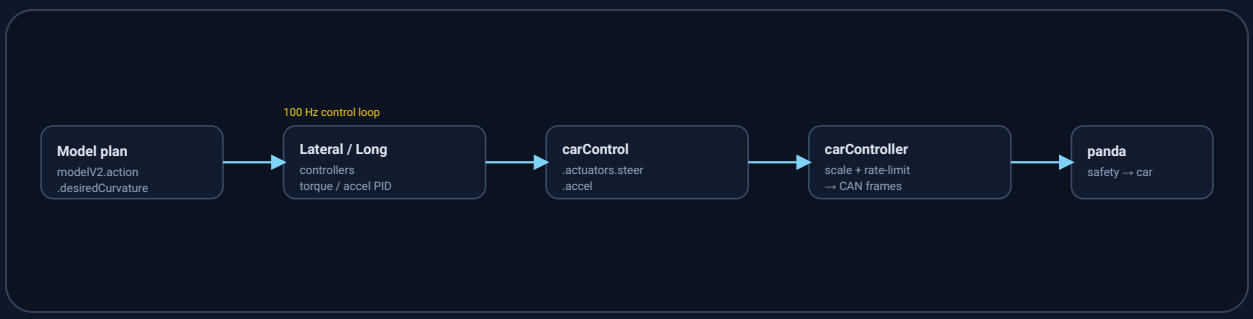
OPENPILOT · CONTROL MODULE

From Planning to Actuation — and porting it to a bus

How openpilot turns a planned trajectory into steering torque, gas & brake — evidence from sunnypilot 0.9.7.1 on a 2020 Honda Civic (Bosch)



The one idea

The model emits a **desired path curvature** (and a speed plan). Everything downstream is **vehicle-specific execution**: turn curvature into a torque the EPS understands, turn the speed plan into an accel command. The plan is transferable; the execution layer is not.

`κ = desiredCurvature [1/m]` `100 Hz` `car: HONDA_CIVIC_BOSCH`

LATERAL

Curvature → steering torque

This unit runs the **torque** controller (param `EnforceTorqueLateral=1`); there is no angle command.

Controller (LatControlTorque)

```
a_lat_des = κ · v² # desired lateral accel
```

```
ff = a_lat / latAccelFactor + friction
   = a_lat / 1.6917 + 0.2546
```

```
steer = PID(err, ff) ∈ [−1, 1] (kp 1.0, ki 0.1)
```

latAccelFactor / friction from the car's torque table. Angle command ≡ 0 in torque mode.

Normalized → CAN torque

```
clip curvature: rate ≤ MAX_LATERAL_JERK/v² (jerk=5)
```

```
apply_steer = −steer × STEER_MAX
STEER_MAX = 4096 → CAN ±4096
```

Sent as `STEERING_CONTROL.STEER_TORQUE`. Final on-bus value = `carOutput.actuatorsOutput.steerOutputCan`.

Angle-controlled cars instead send a steering-angle target; the +/− and scaling differ per brand.

Command vs feedback (what to log)

`carControl.steer` [−1,1] (cmd) → `steerOutputCan` ±4096 (sent) → `carState.steeringTorqueEps` (EPS actual). Compare sign/trend, not raw scale.

LONGITUDINAL

Speed plan → gas / brake

LongControl

```
err = v_target − v_ego (from longitudinalPlan.speeds)
```

```
accel = PID(err, ff=a_target) Bosch clip [−3.5, 2.0]
```

State machine: `off → pid → stopping → starting`. Output = `carControl.actuators.accel` [m/s²].

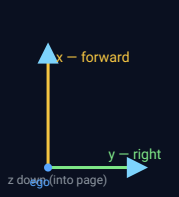
Reality on this car (stock PCM)

- **accel is computed but NOT sent to CAN.** `openpilotLongitudinalControl=False`.
- openpilot only spoofs cruise buttons (`SCM_BUTTONS`) to change the stock ACC set-speed (`CustomStockLong=1`).
- Even with OP long on, Bosch sends `ACC_CONTROL.ACCEL_COMMAND` (car's PCM executes) — no pedal % command.
- Pedals are **measured only**: `carState.gas` (PEDAL_GAS), `carState.brake` (USER_BRAKE) — DBC units, shown as self-calibrated %.

Nidec Hondas differ: they compute gas/brake and send BRAKE_COMMAND + PCM_GAS. The command→actuation mapping is always brand-specific.

COORDINATE SYSTEMS

Frames & transforms



Car / device frame

x fwd, y RIGHT, z down

model path & lanes live here

κ > 0 ⇒ turning right (+y)

Global chain

geodetic ↔ ECEF ↔ NED (lat/lon → local N/E/D)

Curvature ↔ steer angle (bicycle model)

```
κ = 1 / R = yaw_rate / v
```

```
δ_wheel ≈ atan(L · κ)  (L = wheelbase)
```

```
δ_steer ≈ δ_wheel · steerRatio + offset
```

We publish a derived `desSteerAngleDeg` from κ so torque-mode cars still show an angle command (L=2.7 m, steerRatio from liveParameters).

Sign conventions verified in code: torque returns –`output` (left positive); carcontroller "positive when steering right".

VEHICLE-AGNOSTIC OUTPUT

Frenet representation of the trajectory

Two exports that any planner (any vehicle) can consume — no Honda units involved.

1 · relative to lane center (s, d)

```
reference = prob-weighted centerline of inner lanes
d(s) = signed lateral offset (+ = right)
```

Answers "where is the path within the lane." Falls back to the path itself when lanes are missing.

2 · arc-length path profile

```
s, x, y, θ(s), κ(s), v(s), a(s)
```

Pure geometry + motion. **This is the portable trajectory** — feed κ(s), v(s), a(s) to the bus planner directly.

Why it transfers

- κ(s), v(s), a(s) are **physics**, not actuator units.
- Each vehicle maps κ→its own steering, a→its own throttle/brake.
- Exported as CSV via `frenet.to_csv()`.

PORTING

Putting openpilot's planning on a bus

✅ Transfers (vehicle-independent)

- Predicted trajectory / **κ(s), θ(s)**
- Speed & accel profile **v(s), a(s)**
- Frenet (s, d) vs lane center
- The perception → plan idea (curvature + speed target)

🚫 Must be rebuilt for the bus

- κ → steering: **bus kinematics** (long wheelbase, articulation), actuator (hydraulic/steer-by-wire) + its torque/angle scaling
- a → throttle/brake: engine/retarder/air-brake maps
- **Safety layer** (panda equivalent), rate/limit tuning
- Capability envelope: max lateral accel & min turn radius are far tighter

Transferability check (in `frenet_view.py`)

Overlay required $|v^2 \cdot \kappa(s)|$ and κ(s) against the bus envelope (`--bus-wheelbase --bus-max-steer-deg --bus-max-lat-acc`); segments the bus can't follow are shaded red → tells you where openpilot's plan is infeasible for the bus **before** you actuate.